Technical Lesson: File System-Based Digital Forensics
You are a newly hired Junior Digital Forensic Analyst at Sentinel Security Solutions, a cybersecurity firm specializing in incident response and forensic investigations. Your first assignment? Investigate a suspected data deletion on a corporate drive.

The objective: Recover a deleted JPEG file from a disk image, validate its metadata, and confirm its integrity using command-line tools and a Python carving script.

In this lesson, you will use command-line forensic tools to manually carve a JPEG image from a raw disk image by analyzing metadata from the Master File Table (MFT) and extracting it using a Python script. You will then verify the file integrity by comparing its MD5 hash to ensure an accurate recovery.

By the end of this guided walkthrough, you will be able to:

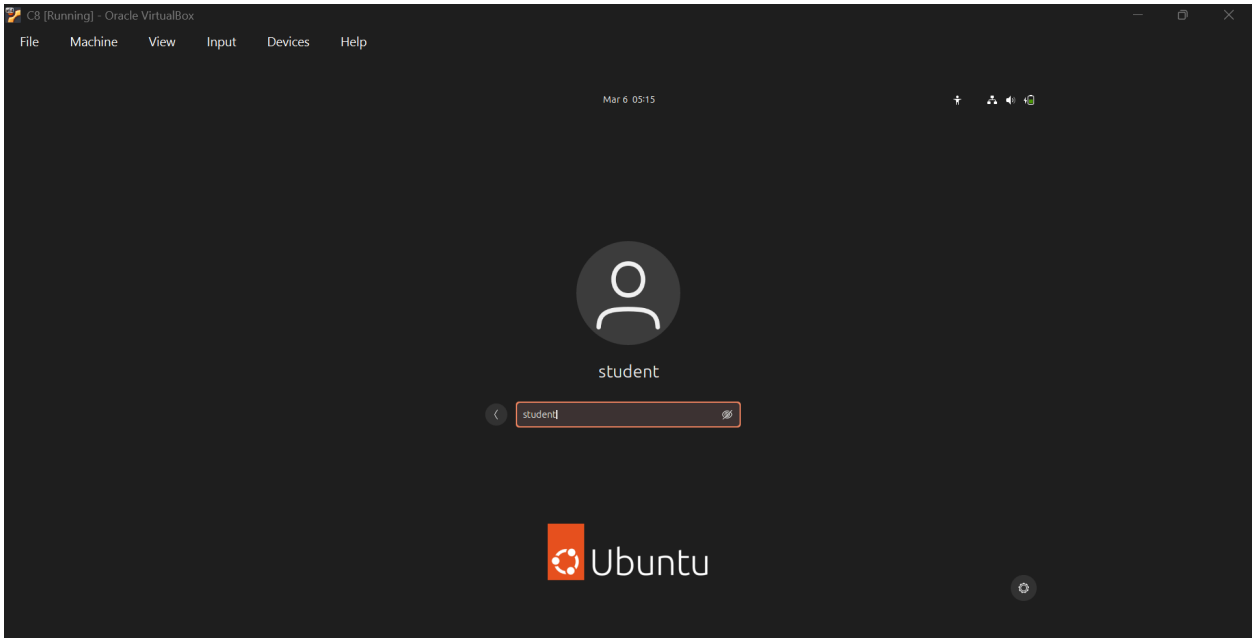
- Use forensic tools to analyze NTFS file system metadata
- Identify deleted files and extract relevant metadata
- Calculate byte offsets for manual recovery
- Recover deleted files using both CLI and Python tools
- Verify forensic integrity using MD5 hashing

Tools and Resources
Tools
- https://drive.google.com/file/d/1vgRgNsmWfugncEfx7ncbHsSy6FHOMI7z/view: All necessary tools and files are preloaded.
- Disk Image File (.dd format): The forensic copy of the NTFS disk.
- Sleuth Kit CLI Tools (fls, fsstat, istat, icat): Used for forensic file system analysis and file recovery.
- Hex Editor (HxD or xxd): Used to analyze file headers and manually verify data integrity.
- Python 3: Used to run a script for automated file carving.

Resources
- https://docs.google.com/document/d/1znfatFau0N6pg_LPBfcX_P9Cp4qxoD8CFatdib4KwZ4/copy: this will prompt you to make a copy.
- Python File Carving Script (Preloaded in VM)

Instructions
Task 1: Set Up the Virtual Environment
Step 1: Launch the Ubuntu VM

1. Open VirtualBox and start the provided Ubuntu VM.
2. Log in using the provided credentials (student/student).



3. Open a terminal and navigate to the forensic materials directory: cd Forensics

```
student@student-flatiron:~$ ls
Desktop  Documents  Downloads  Forensics  Music  Pictures  Public  snap  Templates  Videos
student@student-flatiron:~$ cd Forensics
student@student-flatiron:~/Forensics$ ls
carve_jpg.py  carvepng.py  corrupted_drive.dd  fat32_forensic.dd  file_carver.py  forensic_image.dd  M2_lab.dd
student@student-flatiron:~/Forensics$
```

Step 2: Verify Installed Tools

2. Check if forensic tools are installed:

fls -v

```
student@student-flatiron:~/Forensics$ fls -v
Missing image name
usage: fls [-adDFlhpruvV] [-f fstype] [-i imgtype] [-b dev_sector_size] [-m dir/] [-o imgoffset] [-z ZONE] [-s seconds]
 image [images] [inode]
        If [inode] is not given, the root directory is used
        -a: Display "." and ".." entries
        -d: Display deleted entries only
        -D: Display only directories
        -F: Display only files
        -l: Display long version (like ls -l)
        -i imgtype: Format of image file (use '-i list' for supported types)
        -b dev_sector_size: The size (in bytes) of the device sectors
        -f fstype: File system type (use '-f list' for supported types)
        -m: Display output in mactime input format with
              dir/ as the actual mount point of the image
        -h: Include MD5 checksum hash in mactime output
        -o imgoffset: Offset into image file (in sectors)
        -P pooltype: Pool container type (use '-P list' for supported types)
        -B pool_volume_block: Starting block (for pool volumes only)
        -S snap_id: Snapshot ID (for APFS only)
        -p: Display full path for each file
        -r: Recurse on directory entries
        -u: Display undeleted entries only
        -v: verbose output to stderr
        -V: Print version
        -z: Time zone of original machine (i.e. EST5EDT or GMT) (only useful with -l)
        -s seconds: Time skew of original machine (in seconds) (only useful with -l & -m)
        -k password: Decryption password for encrypted volumes
```

Not present

fsstat -v

```
student@student-flatiron:~/Forensics$ fsstat -v
Missing image name
usage: fsstat [-tvV] [-f fstype] [-i imgtype] [-b dev_sector_size] [-o imgoffset] image
        -t: display type only
        -i imgtype: The format of the image file (use '-i list' for supported types)
        -b dev_sector_size: The size (in bytes) of the device sectors
        -f fstype: File system type (use '-f list' for supported types)
        -o imgoffset: The offset of the file system in the image (in sectors)
        -P pooltype: Pool container type (use '-P list' for supported types)
        -B pool_volume_block: Starting block (for pool volumes only)
        -v: verbose output to stderr
        -V: Print version
        -k password: Decryption password for encrypted volumes
```

Not present

2. Missing a tool? Install it using: sudo apt update && sudo apt install sleuthkit -y

```
student@student-flatiron:~/Forensics$ sudo apt update && sudo apt install sleuthkit -y
[sudo] password for student:
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://us.archive.ubuntu.com/ubuntu noble InRelease
Get:3 http://us.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1,504 kB]
Get:5 http://us.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://us.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,806 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [241 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/main Icons (48x48) [13.4 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/main Icons (64x64) [20.0 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [10.1 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [2,599 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [600 kB]
Get:14 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:15 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 c-n-f Metadata [536 B]
Get:16 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [975 kB]
Get:17 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [218 kB]
Get:18 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.2 kB]
Get:19 http://security.ubuntu.com/ubuntu noble-security/universe Icons (48x48) [49.8 kB]
Get:20 http://security.ubuntu.com/ubuntu noble-security/universe Icons (64x64) [78.3 kB]
Get:21 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [20.6 kB]
Get:22 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Packages [28.8 kB]
Get:23 http://security.ubuntu.com/ubuntu noble-security/multiverse Translation-en [6,732 B]
Get:24 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [212 B]
Get:25 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 c-n-f Metadata [396 B]
Show Apps
```

```
Get:38 http://us.archive.ubuntu.com/ubuntu noble-updates/universe Icons (48x48) [236 kB]
Get:39 http://us.archive.ubuntu.com/ubuntu noble-updates/universe Icons (64x64) [370 kB]
Get:40 http://us.archive.ubuntu.com/ubuntu noble-updates/universe amd64 c-n-f Metadata [32.9 kB]
Get:41 http://us.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Packages [32.1 kB]
Get:42 http://us.archive.ubuntu.com/ubuntu noble-updates/multiverse Translation-en [7,044 B]
Get:43 http://us.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:44 http://us.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 c-n-f Metadata [496 B]
Get:45 http://us.archive.ubuntu.com/ubuntu noble-backports/main amd64 Packages [40.4 kB]
Get:46 http://us.archive.ubuntu.com/ubuntu noble-backports/main Translation-en [9,208 B]
Get:47 http://us.archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [7,308 B]
Get:48 http://us.archive.ubuntu.com/ubuntu noble-backports/main Icons (48x48) [9,536 B]
Get:49 http://us.archive.ubuntu.com/ubuntu noble-backports/main Icons (64x64) [11.3 kB]
Get:50 http://us.archive.ubuntu.com/ubuntu noble-backports/main amd64 c-n-f Metadata [368 B]
Get:51 http://us.archive.ubuntu.com/ubuntu noble-backports/restricted amd64 Components [216 B]
Get:52 http://us.archive.ubuntu.com/ubuntu noble-backports/universe amd64 Packages [29.5 kB]
Get:53 http://us.archive.ubuntu.com/ubuntu noble-backports/universe Translation-en [17.9 kB]
Get:54 http://us.archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [10.5 kB]
Get:55 http://us.archive.ubuntu.com/ubuntu noble-backports/universe Icons (48x48) [23.3 kB]
Get:56 http://us.archive.ubuntu.com/ubuntu noble-backports/universe Icons (64x64) [32.3 kB]
Get:57 http://us.archive.ubuntu.com/ubuntu noble-backports/universe amd64 c-n-f Metadata [1,444 B]
Get:58 http://us.archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Components [212 B]
Fetched 15.8 MB in 26s (616 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
479 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
sleuthkit is already the newest version (4.12.1+dfsg-1.1ubuntu2).
0 upgraded, 0 newly installed, 0 to remove and 479 not upgraded.
student@student-flatiron:~/Forensics$
```

Verify again

- fls -v forensic_image.dd

```
student@student-flatiron:~/Forensics$ fls -v forensic_image.dd
tsk_img_open: Type: 0   NumImg: 1  Img1: forensic_image.dd
aff_open: Error determining type of file: forensic_image.dd
aff_open: Success
Error opening vmdk file
Error checking file signature for vhd file
tsk_img_findFiles: forensic_image.dd found
tsk_img_findFiles: 1 total segments found
raw_open: segment: 0  size: 52428800  max offset: 52428800  path: forensic_image.dd
fsopen: Auto detection mode at offset 0
raw_read: byte offset: 0 len: 65536
raw_read: found in image 0 relative offset: 0 len: 65536
raw_read_segment: opening file into slot 0: forensic_image.dd
ntfs_dinode_lookup: Processing MFT 0
ntfs_proc_attrseq: Processing extended entry for primary entry 0
ntfs_proc_attrseq: Resident Attribute in Type: 16 Id: 0 IdNew: 0 Name:
ntfs_proc_attrseq: Resident Attribute in Type: 48 Id: 2 IdNew: 2 Name:
ntfs_proc_attrseq: Non-Resident Attribute Type: 128 Id: 1 IdNew: 1 Name:   Start VCN: 0
ntfs_make_data_run: Len idx: 0 cur: 19 (13) tot: 19 (13)
ntfs_make_data_run: Off idx: 0 cur: 4 (4) tot: 4 (4)
ntfs_make_data_run: Signed addr_offset: 4 Previous address: 0
ntfs_proc_attrseq: Non-Resident Attribute Type: 176 Id: 3 IdNew: 3 Name:   Start VCN: 0
ntfs_make_data_run: Len idx: 0 cur: 1 (1) tot: 1 (1)
ntfs_make_data_run: Off idx: 0 cur: 2 (2) tot: 2 (2)
ntfs_make_data_run: Signed addr_offset: 2 Previous address: 0
ntfs_dinode_lookup: Processing MFT 3
ntfs_dinode_lookup: Found in offset: 4  size: 19 at offset: 3072
ntfs_dinode_lookup: Entry address at: 19456
ntfs_proc_attrseq: Processing extended entry for primary entry 3
ntfs_proc_attrseq: Resident Attribute in Type: 16 Id: 0 IdNew: 0 Name:
ntfs_proc_attrseq: Resident Attribute in Type: 48 Id: 1 IdNew: 1 Name:
ntfs_proc_attrseq: Resident Attribute in Type: 80 Id: 2 IdNew: 2 Name:
ntfs_proc_attrseq: Resident Attribute in Type: 96 Id: 4 IdNew: 4 Name:
```

- fsstat -v forensic_image.dd

```
student@student-flatiron:~/Forensics$ fsstat -v forensic_image.dd
tsk_img_open: Type: 0   NumImg: 1  Img1: forensic_image.dd
aff_open: Error determining type of file: forensic_image.dd
aff_open: Success
Error opening vmdk file
Error checking file signature for vhd file
tsk_img_findFiles: forensic_image.dd found
tsk_img_findFiles: 1 total segments found
raw_open: segment: 0  size: 52428800  max offset: 52428800  path: forensic_image.dd
fsopen: Auto detection mode at offset 0
raw_read: byte offset: 0 len: 65536
raw_read: found in image 0 relative offset: 0 len: 65536
raw_read_segment: opening file into slot 0: forensic_image.dd
ntfs_dinode_lookup: Processing MFT 0
ntfs_proc_attrseq: Processing extended entry for primary entry 0
ntfs_proc_attrseq: Resident Attribute in Type: 16 Id: 0 IdNew: 0 Name:
ntfs_proc_attrseq: Resident Attribute in Type: 48 Id: 2 IdNew: 2 Name:
ntfs_proc_attrseq: Non-Resident Attribute Type: 128 Id: 1 IdNew: 1 Name:   Start VCN: 0
ntfs_make_data_run: Len idx: 0 cur: 19 (13) tot: 19 (13)
ntfs_make_data_run: Off idx: 0 cur: 4 (4) tot: 4 (4)
ntfs_make_data_run: Signed addr_offset: 4 Previous address: 0
ntfs_proc_attrseq: Non-Resident Attribute Type: 176 Id: 3 IdNew: 3 Name:   Start VCN: 0
ntfs_make_data_run: Len idx: 0 cur: 1 (1) tot: 1 (1)
ntfs_make_data_run: Off idx: 0 cur: 2 (2) tot: 2 (2)
ntfs_make_data_run: Signed addr_offset: 2 Previous address: 0
ntfs_dinode_lookup: Processing MFT 3
ntfs_dinode_lookup: Found in offset: 4  size: 19 at offset: 3072
ntfs_dinode_lookup: Entry address at: 19456
ntfs_proc_attrseq: Processing extended entry for primary entry 3
ntfs_proc_attrseq: Resident Attribute in Type: 16 Id: 0 IdNew: 0 Name:
ntfs_proc_attrseq: Resident Attribute in Type: 48 Id: 1 IdNew: 1 Name:
ntfs_proc_attrseq: Resident Attribute in Type: 80 Id: 2 IdNew: 2 Name:
ntfs_proc_attrseq: Resident Attribute in Type: 96 Id: 4 IdNew: 4 Name:
```

Task 2: Verify the File System & Gather Metadata
Step 1: Identify File System Type

1. Mount the disk image (Read-Only Mode): sudo losetup -Pf forensic_image.dd

```
student@student-flatiron:~/Forensics$ sudo losetup -Pf forensic_image.dd
[sudo] password for student:
student@student-flatiron:~/Forensics$
```

2. List partitions and file system information: sudo fsstat /dev/loop0

```
student@student-flatiron:~/Forensics$ sudo fsstat /dev/loop0
Possible encryption detected (High entropy (8.00))
student@student-flatiron:~/Forensics$
```

3. Identify the file system format (e.g., NTFS, FAT32).
   - **Note:** if the **fsstat** command returns “cannot determine file system type”, check the file type using: file forensic_image.dd. You will see that this disk is NFTS.

```
student@student-flatiron:~/Forensics$ file forensic_image.dd
forensic_image.dd: DOS/MBR boot sector, code offset 0x52+2, OEM-ID "NTFS    ", sectors/cluster 8, Media descriptor 0xf8
, sectors/track 0, dos < 4.0 BootSector (0x80), FAT (1Y bit by descriptor); NTFS, sectors 102399, $MFT start cluster 4,
 $MFTMirror start cluster 6399, bytes/RecordSegment 2^(-1*246), clusters/index block 1, serial number 06fbf04657eaf9a0a
student@student-flatiron:~/Forensics$
```

Step 2: Gather File System Metadata

1. Display file system metadata: fsstat forensic_image.dd
Key Information to Note:

File System Type (e.g., NTFS)
Cluster Size (used for byte offset calculations)
MFT Start Cluster (where file metadata is stored)
Expected Output (Example)

File System Type: NTFS
Cluster Size: 4096 bytes
First Cluster of MFT: 4
Write these down! They are essential for later steps.

```
student@student-flatiron:~/Forensics$  fsstat forensic_image.dd
FILE SYSTEM INFORMATION
--------------------------------------------
File System Type: NTFS
Volume Serial Number: 6FBF04657EAF9A0A
OEM Name: NTFS
Version: Windows XP

METADATA INFORMATION
--------------------------------------------
First Cluster of MFT: 4
First Cluster of MFT Mirror: 6399
Size of MFT Entries: 1024 bytes
Size of Index Records: 4096 bytes
Range: 0 - 65
Root Directory: 5

CONTENT INFORMATION
--------------------------------------------
Sector Size: 512
Cluster Size: 4096
Total Cluster Range: 0 - 12798
Total Sector Range: 0 - 102398

$AttrDef Attribute Values:
$STANDARD_INFORMATION (16)   Size: 48-72   Flags: Resident
$ATTRIBUTE_LIST (32)   Size: No Limit   Flags: Non-resident
$FILE_NAME (48)   Size: 68-578   Flags: Resident,Index
$OBJECT_ID (64)   Size: 0-256   Flags: Resident
$SECURITY_DESCRIPTOR (80)   Size: No Limit   Flags: Non-resident
$VOLUME_NAME (96)   Size: 2-256   Flags: Resident
$VOLUME_INFORMATION (112)   Size: 12-12   Flags: Resident
$DATA (128)   Size: No Limit   Flags:
```

SOLUTION
Key Information:

- **File System Type**: NTFS
- **Cluster Size**: 4096 bytes
- **First Cluster of MFT**: 4

Additional Metadata Information:

- **First Cluster of MFT Mirror**: 6399

- **Size of MFT Entries**: 1024 bytes
- **Size of Index Records**: 4096 bytes
- **Root Directory**: 5

Content Information:

- **Sector Size**: 512 bytes
- **Total Cluster Range**: 0 - 12798
- **Total Sector Range**: 0 - 102398

Task 3: Locate Deleted File Information

The **MFT (Master File Table)** stores information about every file in an NTFS system. Even if a file is deleted, its metadata often remains, allowing us to locate and recover it.

Step 1: List Files in the Disk Image

1. List all files, including **deleted files** (marked with -*): sudo fls -r forensic_image.dd

```
student@student-flatiron:~/Forensics$ sudo fls -r forensic_image.dd
[sudo] password for student:
r/r 4-128-1:    $AttrDef
r/r 8-128-2:    $BadClus
r/r 8-128-1:    $BadClus:$Bad
r/r 6-128-1:    $Bitmap
r/r 7-128-1:    $Boot
d/d 11-144-2:   $Extend
+ r/r 25-144-2: $ObjId:$O
+ r/r 24-144-3: $Quota:$O
+ r/r 24-144-2: $Quota:$Q
+ r/r 26-144-2: $Reparse:$R
r/r 2-128-1:    $LogFile
r/r 0-128-1:    $MFT
r/r 1-128-1:    $MFTMirr
r/r 9-128-2:    $Secure:$SDS
r/r 9-144-3:    $Secure:$SDH
r/r 9-144-4:    $Secure:$SII
r/r 10-128-1:   $UpCase
r/r 10-128-2:   $UpCase:$Info
r/r 3-128-3:    $Volume
-/r * 64-128-2: IMG_3046.jpg
V/V 65: $OrphanFiles
+ -/r * 16:     OrphanFile-16
+ -/r * 17:     OrphanFile-17
+ -/r * 18:     OrphanFile-18
+ -/r * 19:     OrphanFile-19
+ -/r * 20:     OrphanFile-20
+ -/r * 21:     OrphanFile-21
+ -/r * 22:     OrphanFile-22
+ -/r * 23:     OrphanFile-23
student@student-flatiron:~/Forensics$
```

- Look for files marked with **"r/r *" or "d/d *"**, indicating **deleted** files.
- Identify the corresponding **MFT entry number** of a deleted file.
- Example Output: -/r * 64-128-2: IMG_3046.jpg
- This means IMG_3046.jpg was deleted but still has an MFT record (64-128-2).

SOLUTION

iles listed:

1. **r/r 4-128-1**: `$AttrDef`
2. **r/r 8-128-2**: `$BadClus`
3. **r/r 8-128-1**: `$BadClus:$Bad`
4. **r/r 6-128-1**: `$Bitmap`
5. **r/r 7-128-1**: `$Boot`
6. **d/d 11-144-2**: `$Extend` (Directory)
7. **r/r 25-144-2**: `$ObjId:$O`
8. **r/r 24-144-3**: `$Quota:$O`
9. **r/r 24-144-2**: `$Quota:$Q`
10. **r/r 26-144-2**: `$Reparse:$R`
11. **r/r 2-128-1**: `$LogFile`
12. **r/r 0-128-1**: `$MFT`
13. **r/r 1-128-1**: `$MFTMirr`
14. **r/r 9-128-2**: `$Secure:$SDS`
15. **r/r 9-144-3**: `$Secure:$SDH`

16. **r/r 9-144-4**: `$Secure:$SII`

17. **r/r 10-128-1**: `$UpCase`

18. **r/r 10-128-2**: `$UpCase:$Info`

19. **r/r 3-128-3**: `$Volume`

Deleted Files:

1. **-/r * 64-128-2**: `IMG_3046.jpg` (Deleted but has an MFT record)
    - **MFT Entry Number**: 64-128-2
2. **Orphaned Files** (deleted but not fully recovered):
    - **-/r * 16**: `OrphanFile-16`
    - **-/r * 17**: `OrphanFile-17`
    - **-/r * 18**: `OrphanFile-18`
    - **-/r * 19**: `OrphanFile-19`
    - **-/r * 20**: `OrphanFile-20`
    - **-/r * 21**: `OrphanFile-21`
    - **-/r * 22**: `OrphanFile-22`
    - **-/r * 23**: `OrphanFile-23`

Explanation:

- The files marked with `r/r` are regular files, and those with `d/d` are directories.

- The files marked with `-/r *` are **deleted files**, and they still have MFT records, which means they might still be recoverable.

- The **MFT entry number** of a deleted file is crucial for recovery, and in this case, `IMG_3046.jpg` has an MFT entry `64-128-2`.

Step 2: Extract File Metadata & Calculate Byte Offsets

1. Analyze the deleted file using the MFT entry number: sudo istat forensic_image.dd 64-128-2

- File Creation & Deletion Timestamps
- File Size
- Storage Clusters (needed for byte offsets)

Example Output (Key Sections)

Name: IMG_3046.jpg

Created: 2025-03-06 10:33:21

Allocated Size: 10626782 bytes

Storage Clusters: 6912 6913 6914 …

Why does this matter?

- Timestamps can help investigators determine when a file was created or accessed.
- Storage Clusters are needed to calculate byte offsets for manual carving.

SOLUTION

```
student@student-flatiron:~/Forensics$ sudo istat forensic_image.dd 64-128-2
MFT Entry Header Values:
Entry: 64        Sequence: 2
$LogFile Sequence Number: 0
Not Allocated File
Links: 0

$STANDARD_INFORMATION Attribute Values:
Flags: Archive
Owner ID: 0
Security ID: 0  ()
Created:        2025-03-06 10:33:21.359054700 (PST)
File Modified:  2025-03-06 10:33:21.994406000 (PST)
MFT Modified:   2025-03-06 10:33:21.994406000 (PST)
Accessed:       2025-03-06 10:33:21.359054700 (PST)

$FILE_NAME Attribute Values:
Flags: Archive
Name: IMG_3046.jpg
Parent MFT Entry: 5     Sequence: 5
Allocated Size: 10629120        Actual Size: 0
Created:        2025-03-06 10:33:21.359054700 (PST)
File Modified:  2025-03-06 10:33:21.359054700 (PST)
MFT Modified:   2025-03-06 10:33:21.359054700 (PST)
Accessed:       2025-03-06 10:33:21.359054700 (PST)

Attributes:
Type: $STANDARD_INFORMATION (16-0)   Name: N/A   Resident   size: 48
Type: $FILE_NAME (48-3)   Name: N/A   Resident   size: 90
Type: $SECURITY_DESCRIPTOR (80-1)   Name: N/A   Resident   size: 80
Type: $DATA (128-2)   Name: N/A   Non-Resident   size: 10626782  init_size: 10626782
6912 6913 6914 6915 6916 6917 6918 6919
6920 6921 6922 6923 6924 6925 6926 6927
```

Metadata for `IMG_3046.jpg`:

1. **MFT Entry Header Values**:
    - **Entry**: 64
    - **Sequence**: 2
    - **$LogFile Sequence Number**: 0
    - **Links**: 0 (no links)
    - **Not Allocated File** (indicates the file is deleted)
2. **$STANDARD_INFORMATION Attributes**:
    - **Flags**: Archive
    - **Owner ID**: 0
    - **Security ID**: 0

- **Created**: 2025-03-06 10:33:21.359054700 (PST)
- **File Modified**: 2025-03-06 10:33:21.994406000 (PST)
- **MFT Modified**: 2025-03-06 10:33:21.994406000 (PST)
- **Accessed**: 2025-03-06 10:33:21.359054700 (PST)

3. **$FILE_NAME Attribute**:
    - **Flags**: Archive
    - **Name**: `IMG_3046.jpg`
    - **Parent MFT Entry**: 5 (Directory)
    - **Allocated Size**: 10,629,120 bytes
    - **Actual Size**: 0 bytes (likely because the file is deleted)
    - **Created**: 2025-03-06 10:33:21.359054700 (PST)
    - **File Modified**: 2025-03-06 10:33:21.359054700 (PST)
    - **MFT Modified**: 2025-03-06 10:33:21.359054700 (PST)
    - **Accessed**: 2025-03-06 10:33:21.359054700 (PST)
4. **$DATA Attribute** (non-resident):
    - **Size**: 10,626,782 bytes
    - **Storage Clusters**: 6912-6919, 6920-6927, ..., 9504-9505 (these are the storage clusters, which are part of the non-resident data run for the file).
    - This represents the clusters where the file was stored, but since the file has been deleted, the data may still reside in those clusters.

2. To carve the file, we need to calculate the byte offsets.

Start of File:

Cluster Number * Cluster Size.

6912 * 4096 = 28311552

- This result is in decimal format, but we need to convert it to hexadecimal in order to carve. printf "%x\n" 28311552 gives us the “hex” value: (1B00000)

End of File:

Start Of File + File Size

28311552 + 10626782 = 38938334

- Using the printf command, we get the value 25226DE

Task 4: Recover Deleted Files

Now that we have the start and end offsets, we can extract the file.

Method 1: Extract Deleted Files Using CLI Tools

1. The icat tool can recover a file using its **MFT entry number**:

sudo icat forensic_image.dd 64-128-2 > recovered.jpg

2. Verify file integrity using file command:

file recovered_file.jpg

3. If the file is corrupted, analyze the file manually using a hex editor.

Method 2: Automate File Carving with Python

1. Run the script: python3 ~/Forensics/file_carver.py

- Enter the following when prompted:

- File name: forensic_image.dd
- Start Offset: 1B00000
- End Offset: 25226DE
- Output file name: carved.jpg

Task 5: Verify File Integrity (MD5 Hash Comparison)

Even if the file opens, we need to confirm its integrity by checking its MD5 hash against the original.

Step 1: Generate Hashes

1. Run the following commands to compute MD5 checksums:

```
md5sum recovered.jpg carved.jpg
```

- Expected Output (Example):

```
Original:   bfc40fd6b8adf2e7745d833de36b4bc0  IMG_3046.jpg

Recovered:  bfc40fd6b8adf2e7745d833de36b4bc0  recovered.jpg

Carved:     3a54f7b6c95d2a3e88f8a4e07d2a419d  carved.jpg
```

- If the hashes match the original, the file was successfully recovered!
- If the hashes don't match, check the byte offsets used in the carving process. (The recovered and carved should both match the original, which has been given above. The carved showing a different MD5 hash is purely for demonstration purposes here. Both the Python script and the CLI method should carve an identical file.

Considerations

Challenge	**Solution**
The file appears corrupted after recovery	Ensure SOF and EOF bytes fully encompass the file, including footers.
Wrong file type recovered	Verify start and end offsets correspond to a valid file signature.
File does not extract with icat	Try manual extraction using a hex editor.

Decision Points

When should you use manual vs. automated file carving?

How do file system structures impact forensic recovery?

Why does knowing file signatures matter in forensic automation?

Summary: What You Practiced

By following this walkthrough, you have:

- Set up an Ubuntu forensic VM.
- Analyzed an NTFS file system to locate deleted files.
- Recovered files using CLI forensic tools (fls, istat, icat).
- Used a Python script to carve files automatically.

These skills are essential for real-world forensic investigations, where forensic analysts must use both manual and automated approaches to recover digital evidence. In the next Lab, you will extend this knowledge by extracting and analyzing multiple file types to simulate real-world forensic challenges.

Solution

The solution below outlines each step the analyst followed to recover the deleted file, validate its integrity, and document key forensic findings from the NTFS disk image analysis.

Forensic Investigation Overview

Case Summary

As a digital forensic analyst, your objective in this investigation was to recover a deleted image file from a forensic disk image and analyze key metadata to assess its integrity. This analysis focused on the NTFS file system and involved utilizing forensic tools to locate, extract, and validate the recovered file. File recovery is a crucial aspect of digital forensics because it allows analysts to retrieve potential evidence, even when files have been intentionally deleted

File System Metadata Analysis

File System Identified: NTFS

Key Findings:

Cluster Size: 4096 bytes

MFT Start Cluster: 4

Root Directory Entry: 5

The forensic image contained an intact Master File Table (MFT), which retained records of deleted files, allowing for successful recovery.

Why Metadata Matters in Digital Forensics:

Understanding file system metadata is essential because it provides insights into how files are stored, modified, and deleted. In NTFS, even deleted files may still have recoverable metadata, making it possible to analyze timestamps, file paths, and content integrity.

File Recovery Process

Locating Deleted Files

- Forensic Tools Used:

- fls – To list deleted files.
- istat – To analyze file metadata.
- icat – To extract the deleted file.

Deleted File Identified:

- File Name: IMG_3046.jpg
- MFT Entry Number: 64-128-2

Metadata Findings:

- Created: 2025-03-06 10:33:21
- Written: 2025-03-06 10:33:21
- File Size: 10,626,782 bytes
- Storage Clusters: 6912 - 9506

Analyzing Suspicious Timestamps

- No anomalies were detected in the timestamps for the deleted file in this investigation. However, timestamps are crucial in forensic investigations, as they can indicate whether a file has been manipulated, potentially affecting the credibility of forensic evidence.

Byte Offset Calculation for File Carving

- Using metadata from istat, the byte offsets were calculated as follows:

  - Start of File (SOF) Calculation:

- Cluster Start: 6912
- Cluster Size: 4096 bytes
- SOF (Decimal): 6912 * 4096 = 28,311,552
- SOF (Hexadecimal):1B00000

End of File (EOF) Calculation:

- File Size: 10,626,782 bytes
- EOF (Decimal): 28,311,552 + 10,626,782 = 38,938,334

- EOF (Hexadecimal):25226DE

Note: A comparison of NTFS vs. FAT32 recovery processes will be addressed in the lab.

File Recovery and Integrity Validation

Recovering Files Using CLI and Python

CLI Recovery Method: sudo icat forensic_image.dd 64-128-2 > recovered.jpg

Result: The image file was successfully recovered and was accessible.

Python Recovery Method: python3 ~/Forensics/file_carver.py

Offsets Used:

- SOF: 1B00000
- EOF: 25226DE
- Output file name: carved.jpg
- Result: The Python script successfully extracted the file.

MD5 Hash Verification

To confirm data integrity, MD5 hashes were generated for the recovered files:

md5sum recovered.jpg carved.jpg

File	MD5 Hash
Original (Provided)	bfc40fd6b8adf2e7745d833de36b4bc0
CLI-Recovered	bfc40fd6b8adf2e7745d833de36b4bc0
Python-Carved	bfc40fd6b8adf2e7745d833de36b4bc0

Reflection and Forensic Insights

Challenges and Solutions

Challenges Encountered:

- No significant technical issues were encountered during the NTFS file recovery process.
- However, a common challenge in forensic analysis is miscalculating byte offsets, which can result in extracting corrupt or incomplete files.
- Another potential issue is forgetting to convert decimal byte offsets into hexadecimal, leading to incorrect file carving.

Solutions Applied:

- Ensured metadata was thoroughly analyzed before extracting the file.
- Carefully verified byte offset calculations before running the carving script.
- Used both CLI (icat) and Python-based recovery to cross-validate the recovered file's integrity.

Comparing NTFS vs. FAT32 File Recovery

(Note: full comparison will be completed in the assignment lab)

- NTFS uses the Master File Table (MFT) for tracking files
- The recovery process may vary based on how each system manages deleted files and timestamps.
- Differences in metadata storage may affect the ease of forensic analysis.

Implications for Digital Forensics

- Why is metadata analysis essential? It helps determine file authenticity and detect tampering.
- How do timestamps contribute to forensic evidence? They can provide critical insights into when a file was created, accessed, or modified.
- How does automated file carving enhance forensic efficiency? It enables large-scale recovery while minimizing human error, making investigations more efficient.